

Golang 混合写屏障机制详解

一、背景问题

在三色并发标记过程中，用户代码可能随时修改指针，导致以下问题：

黑色对象 → 新建的白色对象

如果不加处理，这个白色对象会被错误地当作垃圾回收，导致程序崩溃。

二、Go GC 的演进

版本	方案	问题
Go 1.3	标记清除法	需要 STW，效率极低
Go 1.5	三色标记 + 插入/删除屏障	需要重新扫描栈（STW）
Go 1.8+	三色标记 + 混合写屏障	几乎消除 STW

三、混合写屏障的 4 条规则

规则详解

规则	说明	目的
规则 1	GC 开始时，栈上所有对象标记为黑色	栈无需二次扫描，避免 STW
规则 2	GC 期间栈上新对象直接为黑色	栈对象不会被误回收
规则 3	删除引用时，被删除对象标记为灰色	防止丢失被引用的对象
规则 4	添加引用时，被添加对象标记为灰色	防止黑色→白色引用丢失

四、混合写屏障 = 插入屏障 + 删除屏障

插入屏障：黑引用白时，把白染灰（强三色不变式）
删除屏障：断开灰→白引用时，把白染灰（弱三色不变式）

为什么栈上不开启写屏障？

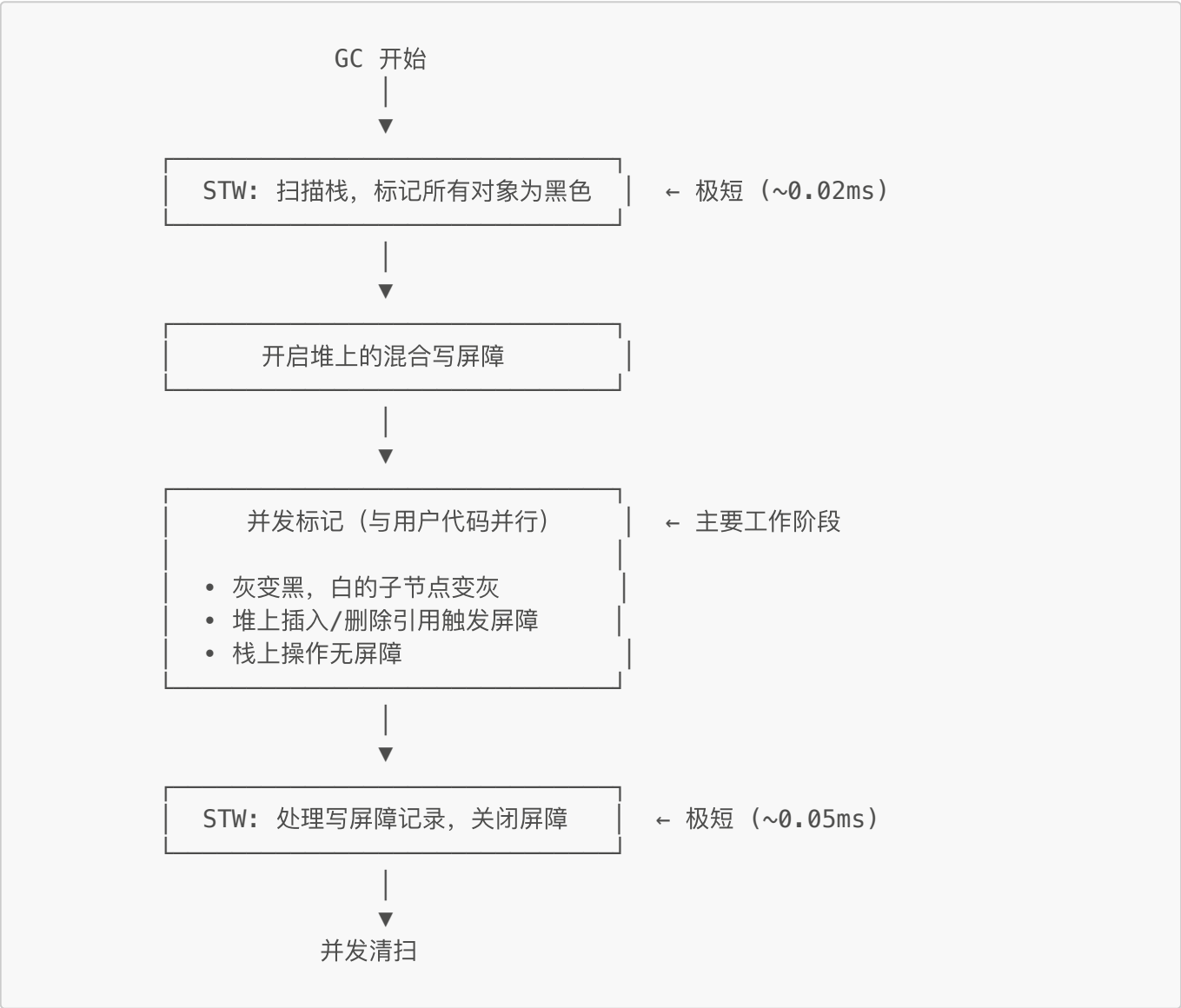
- 栈操作极频繁，每次操作都触发写屏障会严重影响性能
- 解决方案：
 - 栈对象在 GC 开始时统一扫描标记为黑色

- 只在堆上开启写屏障
- GC 结束时对栈做极短 STW 重扫

五、三色不变式

不变式	含义	保障
强三色不变式	黑色对象不会直接指向白色对象	插入屏障保证
弱三色不变式	黑色对象指向的白色对象，必须被其他灰色对象保护	删除屏障保证

六、混合写屏障工作流程图



七、代码层面理解

写屏障的伪代码

```
// 混合写屏障的简化逻辑
func writeBarrier(slot *unsafe.Pointer, new unsafe.Pointer) {
    // 规则3: 被删除的对象标记为灰色
    old := *slot
    if old != nil {
        shade(old) // 把旧对象染灰
    }

    // 规则4: 被添加的对象标记为灰色
    if new != nil {
        shade(new) // 把新对象染灰
    }

    // 执行指针更新
    *slot = new
}
```

八、对比总结

特性	Go 1.5 插入屏障	Go 1.5 删除屏障	Go 1.8 混合写屏障
栈扫描	需要二次扫描 (STW)	需要二次扫描 (STW)	只扫描一次
回收精度	高	低 (下一轮回收)	高
STW 时间	较长	较长	极短
性能开销	中等	中等	较低

九、面试要点速记

- 1. 混合写屏障解决了什么问题？
 - 并发标记时，防止用户代码修改指针导致对象丢失
- 2. 为什么叫"混合"？
 - 结合了插入屏障和删除屏障的优点
- 3. 栈上为什么不开启写屏障？
 - 栈操作频繁，开启屏障会严重影响性能
- 4. 如何保证正确性？
 - 栈对象在 GC 开始时统一标记为黑色
 - 堆上的写屏障保证三色不变式
- 5. STW 时间为什么变短了？
 - 不需要重新扫描栈，只需处理写屏障记录